

Struttura di un programma C e compilazione

Marco Cerbella

Lezione 4 – 29 settembre 2026

1 Struttura di un programma

Gli elementi costitutivi di un programma C sono le *funzioni*, che possono chiamarsi a vicenda.

- Ogni funzione ha uno scopo preciso; le funzioni *non* si possono annidare.
- Una funzione contiene istruzioni eseguite in sequenza; le istruzioni si raggruppano in *blocchi*.
- Si possono usare funzioni della libreria standard (`printf`) o scriverne di proprie.
- Ogni programma deve definire almeno la funzione `main()`: è la prima funzione eseguita ed è il livello principale di controllo.

2 Hello World e compilazione con gcc

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Hello World\n");
5     return 0;
6 }
```

Salvato il codice in `test.c`, da terminale si compila (creando l'eseguibile `test`) e poi si esegue:

```
gcc -o test test.c
./test
```

Osservazione. Per il corso e per l'esame si compila *solo con gcc* da terminale (anche se si usa un editor che ha un terminale integrato). Senza `-o` il file eseguibile si chiama `a.out`.

3 Input e output: printf e scanf

Esempio (Somma di due interi).

```
1 #include <stdio.h>
2 int main() {
3     int firstNumber, secondNumber, sumOfTwoNumbers;
```

```

4
5     printf("Enter two integers: ");
6
7     // gli interi inseriti vengono memorizzati con scanf()
8     scanf("%d %d", &firstNumber, &secondNumber);
9
10    // la somma viene salvata in sumOfTwoNumbers
11    sumOfTwoNumbers = firstNumber + secondNumber;
12
13    // stampa della somma
14    printf("%d + %d = %d", firstNumber, secondNumber, sumOfTwoNumbers);
15
16    return 0;
17 }

```

```

Enter two integers: 12
11
12 + 11 = 23

```

Esempio (Quoziente e resto).

```

1  #include <stdio.h>
2  int main() {
3      int dividend, divisor, quotient, remainder;
4
5      printf("Enter dividend: ");
6      scanf("%d", &dividend);
7
8      printf("Enter divisor: ");
9      scanf("%d", &divisor);
10
11     quotient = dividend / divisor;    // quoziente
12     remainder = dividend % divisor;   // resto
13
14     printf("Quotient = %d\n", quotient);
15     printf("Remainder = %d", remainder);
16
17     return 0;
18 }

```

```

Enter dividend: 25
Enter divisor: 4
Quotient = 6
Remainder = 1

```

Osservazione. %d è il segnaposto per un intero; in scanf le variabili vanno precedute da &.

4 Programmazione procedurale e modulare

Definizione. La *programmazione procedurale* è un paradigma, derivato dalla programmazione strutturata, basato sul concetto di chiamata di procedura. Le procedure (o routine, sottoprogrammi, funzioni) contengono una serie di passi di calcolo da eseguire. Il C è imperativo e procedurale: le istruzioni sono raggruppate in funzioni.

Esempio. `mainfile.c`

```
1  #include <stdio.h>
2  void myPrintHello();
3
4  int main() {
5      myPrintHello();
6      return(0);
7  }
8
9  void myPrintHello(void) {
10     printf("Hello!\n");
11     return;
12 }
```

Definizione. La *programmazione modulare* è una tecnica di progettazione che separa le funzionalità di un programma in *moduli* indipendenti e intercambiabili, ciascuno contenente tutto il necessario per realizzare un solo aspetto della funzionalità.

Ad esempio il codice si può dividere in file diversi.

`mainfile.c`:

```
1  int main() {
2      myPrintHello();
3      return(0);
4  }
```

`hello.c`:

```
1  #include <stdio.h>
2
3  void myPrintHello(void) {
4      printf("Hello!\n");
5      return;
6  }
```

e si compilano insieme:

```
gcc -o executable mainfile.c hello.c
```